

1

1

$$z = \sum w_i x_i + b$$

$$\rightarrow z = x_1 + x_2 - 0.6$$

$$z \geq 0 \leftarrow y = 1$$

$$z < 0 \leftarrow y = 0$$

$$\sigma(0,0) = 0$$

$$\sigma(0,1) = 1$$

$$\sigma(1,0) = 1$$

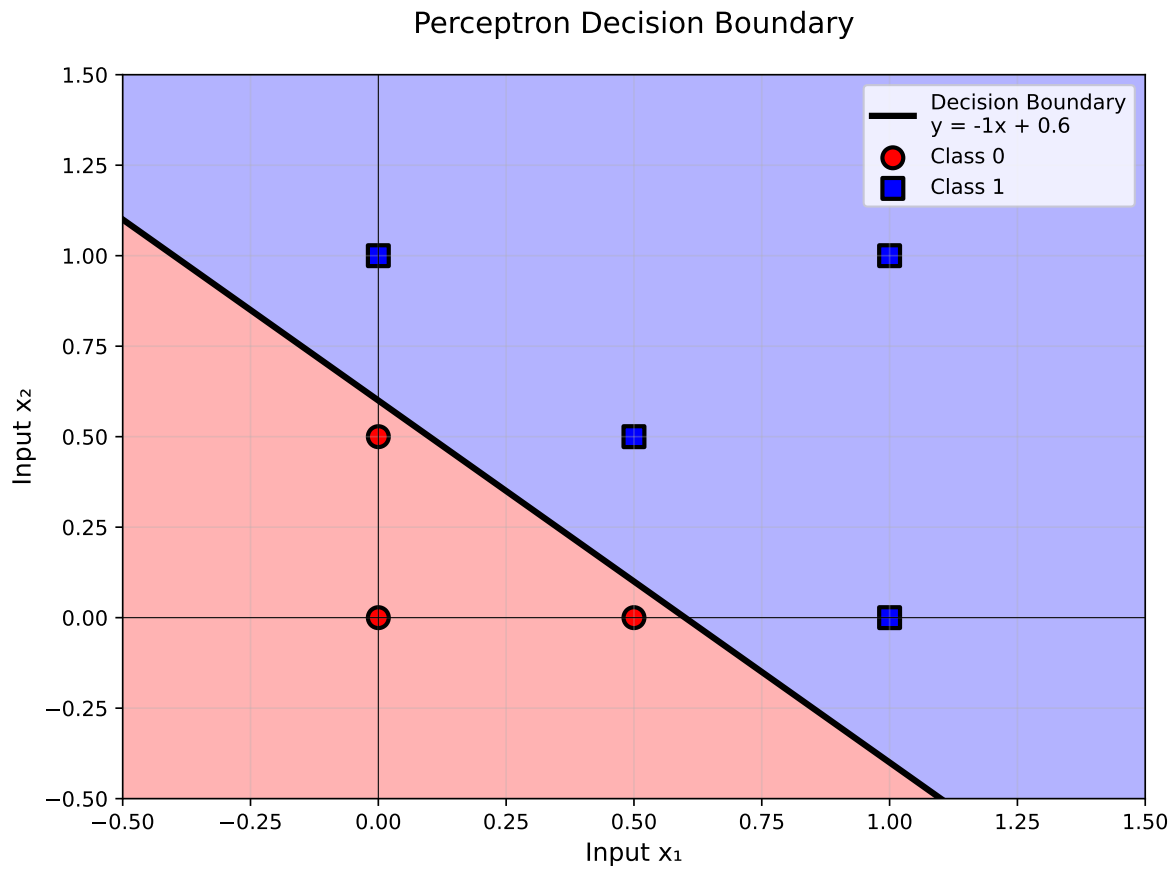
$$\sigma(1,1) = 1$$

$$\sigma(0,0.5) = 0$$

$$\sigma(0.5,0) = 0$$

$$\sigma(0.5,0.5) = 1$$

2



3

This would represent the logical OR function because the perceptron is 0 iff $x_1 = 0 \wedge x_2 = 0$

2

1 NAND

Assuming only x_1 and x_2 we get the following:

$$w_1 = -1, w_2 = -1, b = 1.5$$

We can get these weights and biases as long as they follow these inequalities:

1. $w_1 + w_2 + b < 0$
2. $w_1 + b \geq 0$
3. $w_2 + b \geq 0$
4. $b \geq 0$

For an n input NAND operator:

$$\forall i (w_i = -1), b = n - 0.5$$

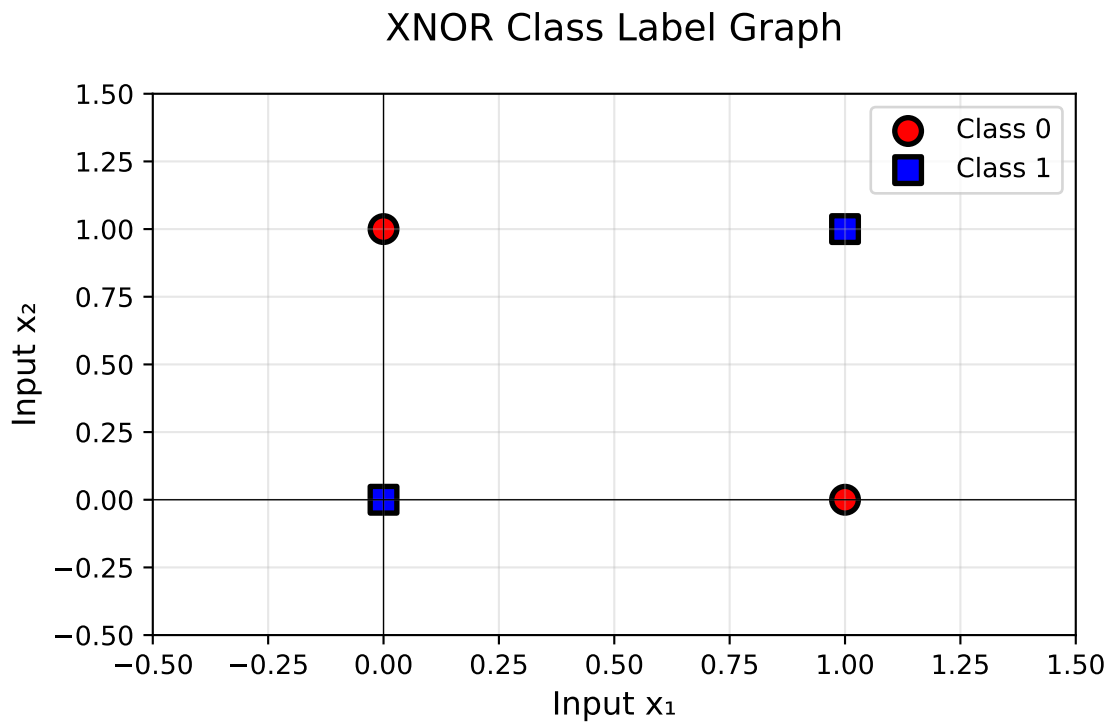
2 NOT

Assuming only x_1 we get the following: $w_1 = -1, b = 0.5$

1. $w_1 + b < 0$
2. $b \geq 0$

3

1



2

No single straight line can separate the two classes because the classes are in opposite corners as shown above.

3

No it is not possible to construct a perceptron for the two input XOR. We know that the weights and biases must follow the below inequalities:

1. $w_1 + w_2 + b < 0$
2. $w_1 + b \geq 0$
3. $w_2 + b \geq 0$

4. $b < 0$

From 4, we know b must be negative. Using this information we know that w_1, w_2 must both be positive to make 2,3 true.

Now looking at number 1, $w_1 + w_2 + b < 0$, we can substitute inequalities 2,3 in to get:

$$w_1 + w_2 + b \geq -b - b + b$$

$$\rightarrow -b - b + b = -b$$

From inequality 4, we know that $-b > 0$ so we know that $w_1 + w_2 + b \geq 0$ which creates a contradiction because inequality 1 says that $w_1 + w_2 + b < 0$.

Therefore it is not possible to create a perceptron for the XOR logical operation.

4

x	y
1	0
2	0
3	1
4	1

1

Initial

$$w_0 = 0 \quad b_0 = 0.1 \quad \eta = 0.1$$

First Pass

$$\delta w = \eta(0 - 1)(1) = -0.1 \quad \delta b = \eta(0 - 1) = -0.1 \rightarrow w_1 = -0.1 \rightarrow b_1 = 0$$

Second Pass

$$\delta w = \eta(0 - 0)(2) = 0 \quad \delta b = \eta(0 - 0) = 0 \rightarrow w_2 = -0.1 \rightarrow b_2 = 0$$

Third Pass

$$\delta w = \eta(1 - 0)(3) = 0.3 \quad \delta b = \eta(1 - 0) = 0.1 \rightarrow w_3 = 0.2 \rightarrow b_3 = 0.1$$

Fourth Pass

$$\delta w = \eta(1 - 1)(4) = 0 \quad \delta b = \eta(1 - 1) = 0 \rightarrow w_4 = 0.2 \rightarrow b_4 = 0.1$$

2

$$w = 0.1, b = 0.1$$

$$z = [0.2, 0.3, 0.4, 0.5]$$

$$L(w, b) = \frac{1}{n} \sum (y_i - \sigma'(z_i))^2$$

$$L(w, b) = \frac{1}{4}(0.2^2 + 0.3^2 + 0.6^2 + 0.5^2)$$

$$L(w, b) = \frac{1}{4}(0.74)$$

$$L(w, b) = 0.185$$

3

$$\nabla_w L(w, b)$$

$$\frac{dL}{dw} = -\frac{2}{n} \sum (y_i - \sigma'(z_i))x_i$$

$$\frac{dL}{dw} = -\frac{1}{2}(-0.2(1) - 0.3(2) + 0.6(3) + 0.5(4))$$

$$\frac{dL}{dw} = -\frac{1}{2}(3)$$

$$\frac{dL}{dw} = 1.5$$

$$\frac{dL}{db} = -\frac{2}{n} \sum (y_i - \sigma'(z_i))$$

$$\frac{dL}{db} = -\frac{1}{2}(-0.2 - 0.3 + 0.6 + 0.5)$$

$$\frac{dL}{db} = -\frac{1}{2}0.6$$

$$\frac{dL}{db} = 0.3$$

Thus after one epoch (with $\eta = 0.1$):

$$w = -0.5, b = 0.07$$

4

Order of inputs does not matter for the gradient descent algorithm because the gradient descent algorithm uses consistent weights and biases for each input in a single epoch. In other words, since we only update the weights and biases AFTER performing gradient descent (and apply the same weight to each input in the gradient descent function) the order of inputs does not matter. This is different to the perceptron model which updates the weights and biases within a single epoch (and each input could have different weights and biases applied), thus the order matters for the perceptron model.

5

1

Input Features

preg = Number of times pregnant

plas = Plasma glucose concentration a 2 hours in an oral glucose tolerance test

pres = Diastolic blood pressure (mm Hg)

skin = Triceps skin fold thickness (mm)

test = 2-Hour serum insulin (μ U/ml)

mass = Body mass index ($\text{weight in kg}/(\text{height in m})^2$)

pedi = Diabetes pedigree function

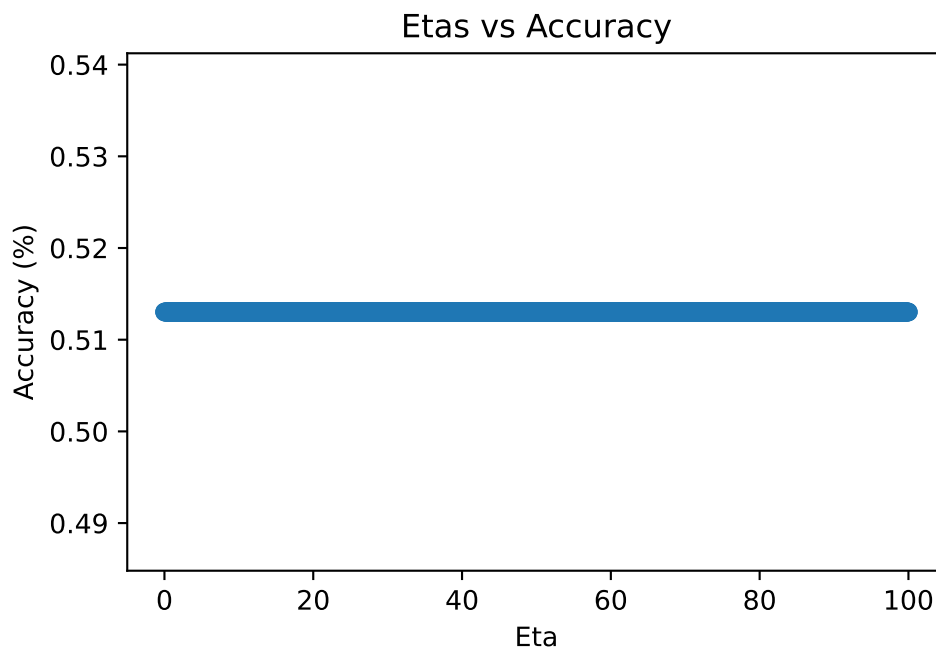
age = Age (years)

Output Classes

1: tested positive for diabetes

0: tested negative for diabetes

2



Here we see that all η 's had the same accuracy of 51.302% after 1000 epochs.

3

C:\Users\aidan\AppData\Roaming\Python\Python313\site-packages\sklearn\linear_model_stochastic

Maximum number of iteration reached before convergence. Consider increasing max_iter to impr

C:\Users\aidan\AppData\Roaming\Python\Python313\site-packages\sklearn\linear_model_stochastic

Maximum number of iteration reached before convergence. Consider increasing max_iter to impr

C:\Users\aidan\AppData\Roaming\Python\Python313\site-packages\sklearn\linear_model_stochastic

Maximum number of iteration reached before convergence. Consider increasing max_iter to impr

C:\Users\aidan\AppData\Roaming\Python\Python313\site-packages\sklearn\linear_model_stochastic

Maximum number of iteration reached before convergence. Consider increasing max_iter to impr

C:\Users\aidan\AppData\Roaming\Python\Python313\site-packages\sklearn\linear_model_stochastic

Maximum number of iteration reached before convergence. Consider increasing max_iter to improve

C:\Users\aidan\AppData\Roaming\Python\Python313\site-packages\sklearn\linear_model_stochastic

Maximum number of iteration reached before convergence. Consider increasing max_iter to improve

C:\Users\aidan\AppData\Roaming\Python\Python313\site-packages\sklearn\linear_model_stochastic

Maximum number of iteration reached before convergence. Consider increasing max_iter to improve

C:\Users\aidan\AppData\Roaming\Python\Python313\site-packages\sklearn\linear_model_stochastic

Maximum number of iteration reached before convergence. Consider increasing max_iter to improve

C:\Users\aidan\AppData\Roaming\Python\Python313\site-packages\sklearn\linear_model_stochastic

Maximum number of iteration reached before convergence. Consider increasing max_iter to improve

